

Lernziele HFO

- **Standard-Interfaces erkennen und nutzen**
- **Eigene HOFs implementieren (die Lambdas *empfangen*)**
- **Übergebene Logik anwenden**
- **Eigene FunctionalInterfaces definieren**
- **Datenflüsse verstehen (map/filter)**

1. Standard-Interfaces erkennen und nutzen:

"Du musst die wichtigsten java.util.function-Interfaces (besonders Predicate, Consumer, Function/UnaryOperator, Supplier) und ihre *einzige* Methode (test, accept, apply, get) erkennen."

"Du musst aus einem Unit-Test ablesen können, welches Interface das richtige ist (z.B. movie -> boolean ist ein Predicate)."

2. Eigene HOFs implementieren (die Lambdas *empfangen*):

"Du musst eine eigene Methode schreiben können (wie findByCategory), die ein Functional Interface (z.B. Predicate<Movie> criteria) als Parameter entgegennimmt."

3. Übergebene Logik anwenden:

"Du musst *innerhalb* deiner HOF die korrekte Methode des Lambda-Parameters aufrufen (z.B. criteria.test(movie) in einer if-Abfrage oder action.accept(movie) in einer Schleife)."

4. Eigene FunctionalInterfaces definieren:

"Für Anwendungsfälle, die nicht von Standard-Interfaces abgedeckt werden (wie der Combiner mit T, T -> R), musst du ein eigenes Interface mit @FunctionalInterface und der korrekten Methodensignatur erstellen können."

5. Datenflüsse verstehen (map/filter):

"Du musst HOFs implementieren können, die Daten transformieren (applyToEach) oder filtern (applyToEachIf) und das Ergebnis als *neue* Collection zurückgeben, statt die Originaldaten zu verändern."

Lernziele Callbacks

- **Das Observer Pattern erkennen**
- **Den Publisher implementieren**
- **Das Event auslösen**
- **Semantische Interfaces nutzen**
- **Callback-Code testen**

1. Das Observer Pattern erkennen:

"Du musst das Grundmuster erkennen: Ein 'Publisher' (Subjekt) verwaltet eine Liste von 'Subscribern' (Beobachtern) und benachrichtigt sie über ein 'Event'."

2. Den Publisher implementieren:

"Du musst wissen, wie man einen Publisher baut:

Eine Collection (z.B. List oder Set) für die Subscriber (Callbacks) anlegen.

Eine subscribe-Methode implementieren, die einen neuen Callback zur Liste hinzufügt.

Eine notify/publish-Methode implementieren, die über die Liste iteriert und *jeden* Callback aufruft."

3. Das Event auslösen:

"Du musst erkennen, *wo* das Event stattfindet, und dort deine notify-Methode aufrufen (z.B. am Ende von upload oder am Ende von setName/setId)."

4. Semantische Interfaces nutzen:

"Du musst verstehen, wann man Standard-Interfaces (Consumer) oder (wie bei BigBrother oder dem Waffeleisen) eigene,

semantisch benannte Functional Interfaces
(NameChangedCallback) verwendet."

5. Callback-Code testen:

"Du musst wissen, wie man void-Methoden testet, die Callbacks auslösen: indem du ein Test-Lambda mit einer Assertion als Subscriber registrierst und *danach* die Methode (Act) aufrufst (das 'Arrange-Assert-Act'-Muster)."